

Deep learning part III

Olivier Caelen

Attention and Transformer Networks

Attention Mechanism

Attention is a neural architecture that mimics the retrieval of a **value** v_i for a **query** q based on a **key** k_i in a database.

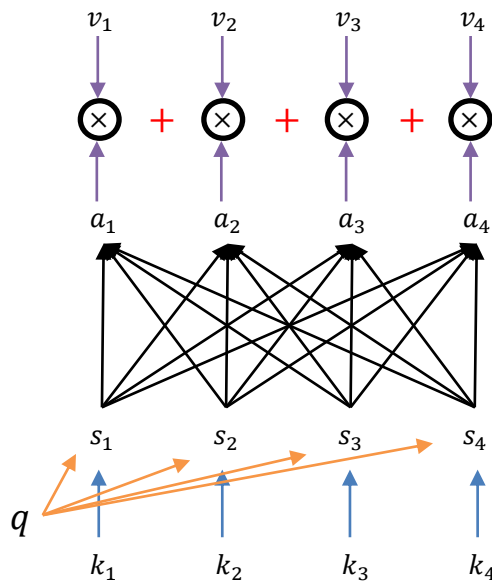
$$\text{attention}(q, \{k_i\}, \{v_i\}) = \sum_i \text{similarity}(q, k_i) \times v_i$$

Query	Key 1	Value 1
	Key 2	Value 2
	Key 3	Value 3
	...	...
	Key T	Value T

In a database, I search for the key corresponding to the query and return the value associated with the key.

Question for you: For each of the following elements, say if there are: scales or vectors

- $q, k_i, s_i, a_i, v_i, \text{attention_value}$



$$\text{Attention value} = \sum_i a_i v_i = c$$

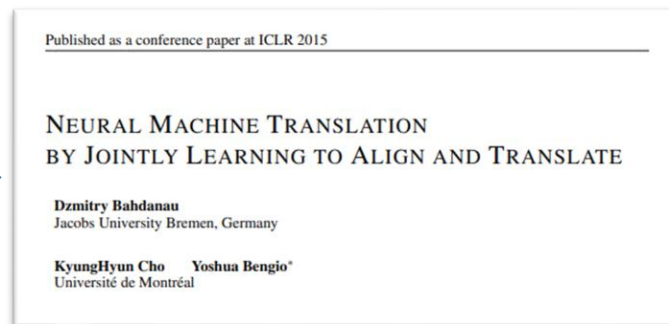
$$a_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}, \quad \text{softmax}$$

$$s_i = \begin{cases} q^T k_i, \text{ dot product} \\ \frac{q^T k_i}{\sqrt{d}}, \text{ scaled dot product} \\ q^T W k_i, \text{ general dot product} \\ \dots \end{cases}$$

Where d is the dimensionality of each key.

Attention Mechanism

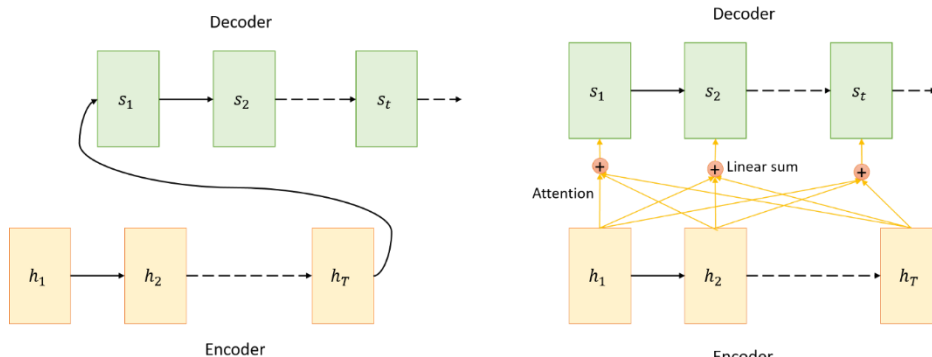
- ‘*Attention*’ is a general scheme where we have a query q , we have some keys $\{k_i\}$, and we produce an output which is a linear combination of some values $\{v_i\}$ where the weights of the linear combination come from some notion of similarity between our query q and the keys $\{k_i\}$.
- Attention Mechanism for sequence modelling was first used in the paper: *Neural Machine Translation by jointly learning to align and translate* in 2015.



A potential issue with this encoder–decoder approach is that a neural network needs to be able to compress all the necessary information of a source sentence into a fixed-length vector. This may make it difficult for the neural network to cope with long sentences, especially those that are longer than the sentences in the training corpus. Cho *et al.* (2014b) showed that indeed the performance of a basic encoder–decoder deteriorates rapidly as the length of an input sentence increases.

Attention mechanism in RNN

Unlike classic encoder-decoder approach, attention mechanism in RNN helps to look at all hidden states from encoder sequence for making predictions.



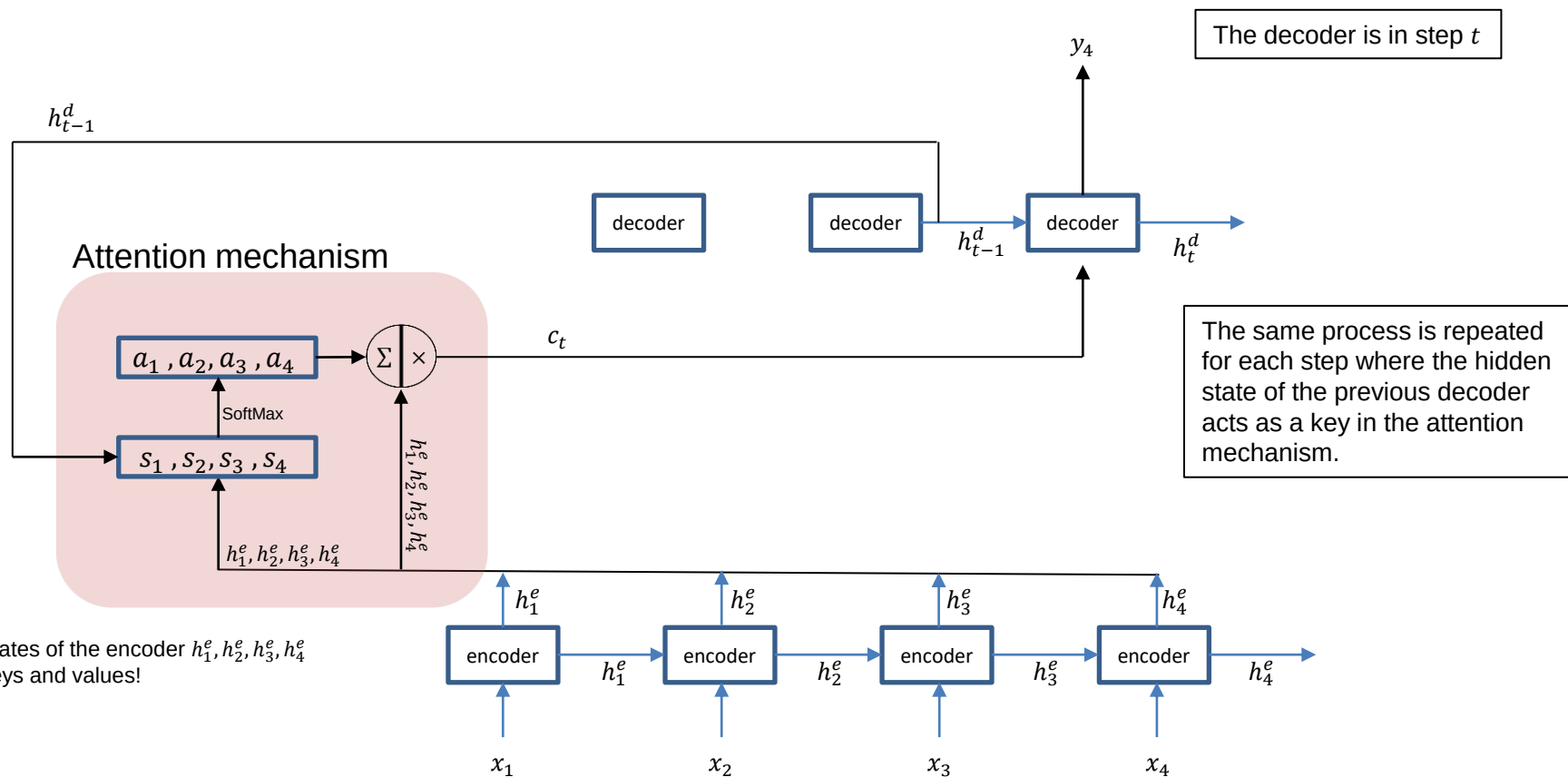
(a) Vanilla Encoder Decoder Architecture

(b) Attention Mechanism

Credit: A simple overview of RNN, LSTM and Attention Mechanism (<https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b>)

- In a classical encoder-decoder architecture, the decoder is supposed to make predictions by only looking at the final output of the encoder stage which contains packed information.
- Attention based architecture connect every hidden state from each encoder node at every time step and then makes predictions after deciding which one is more informative.

Attention mechanism in RNN



Transformers

Issues with RNN

- Encoding bottleneck
- Slow, no parallelization
- Not (really) long memory



Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

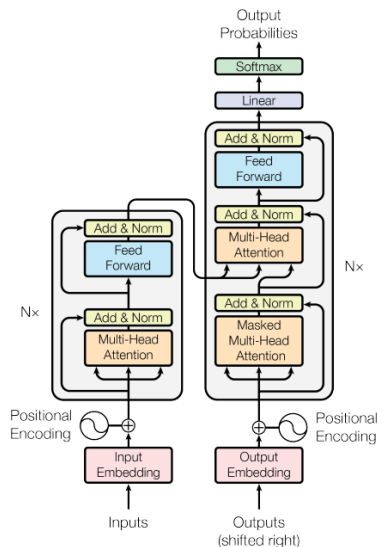
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

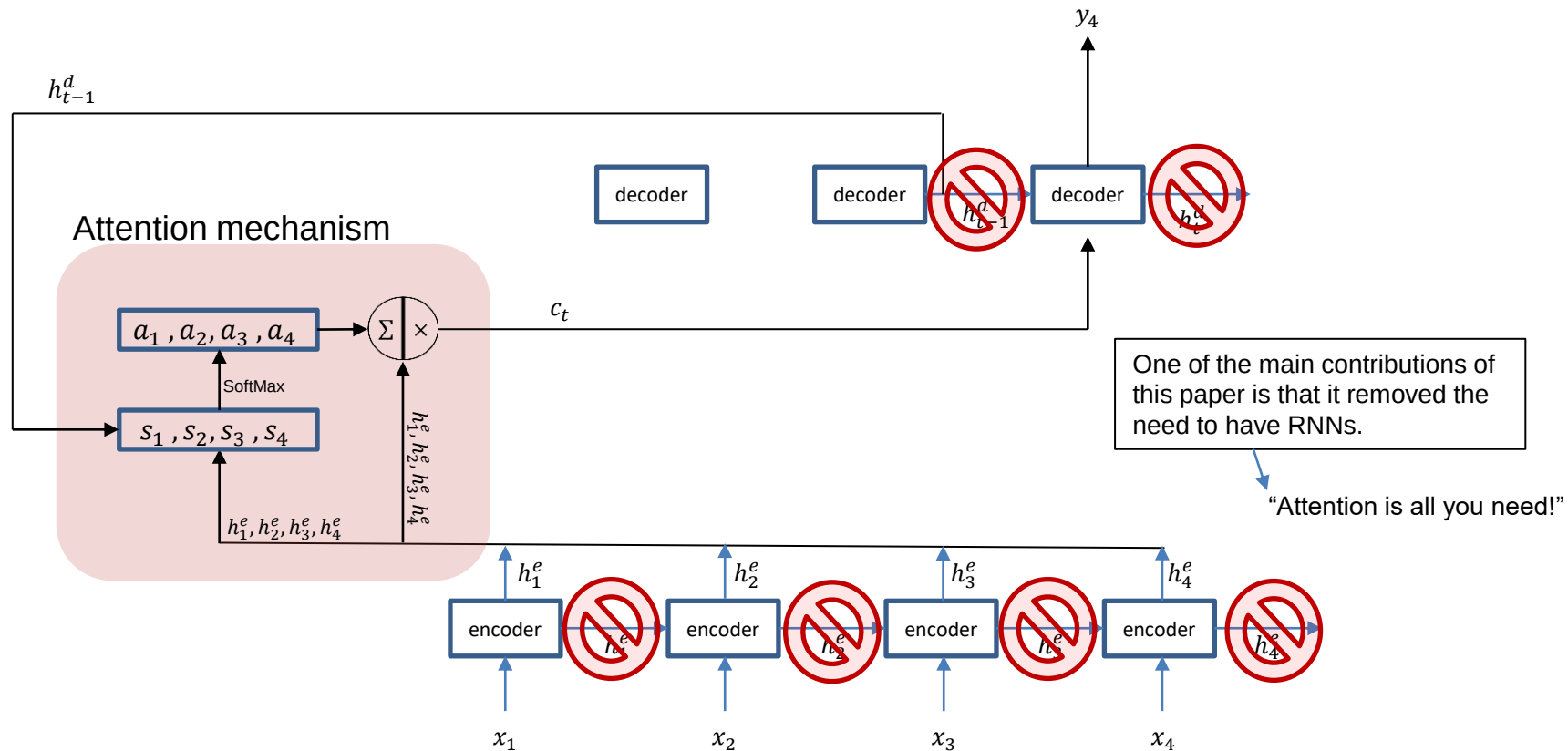
Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

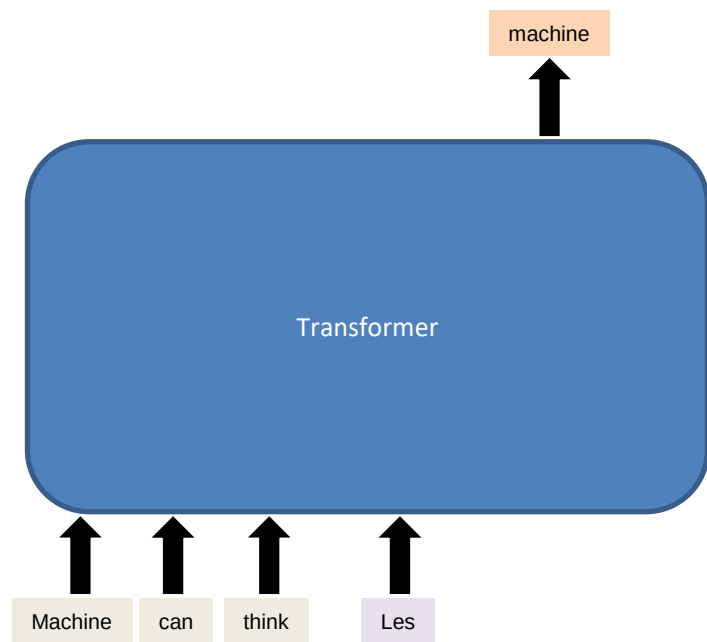
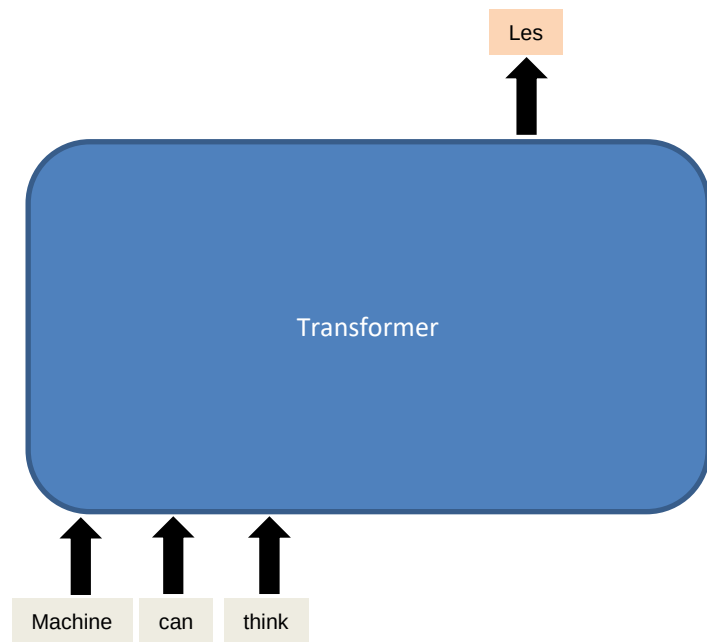
Vaswani et al., NeurIPS, 2017



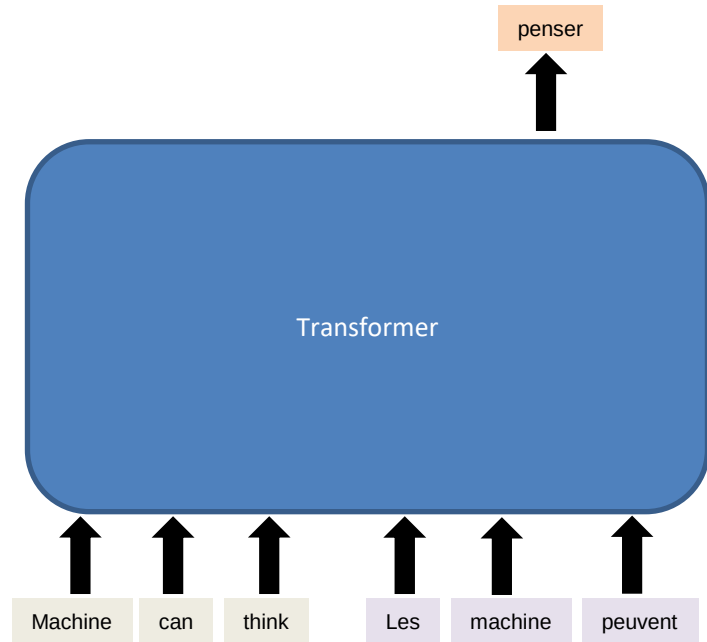
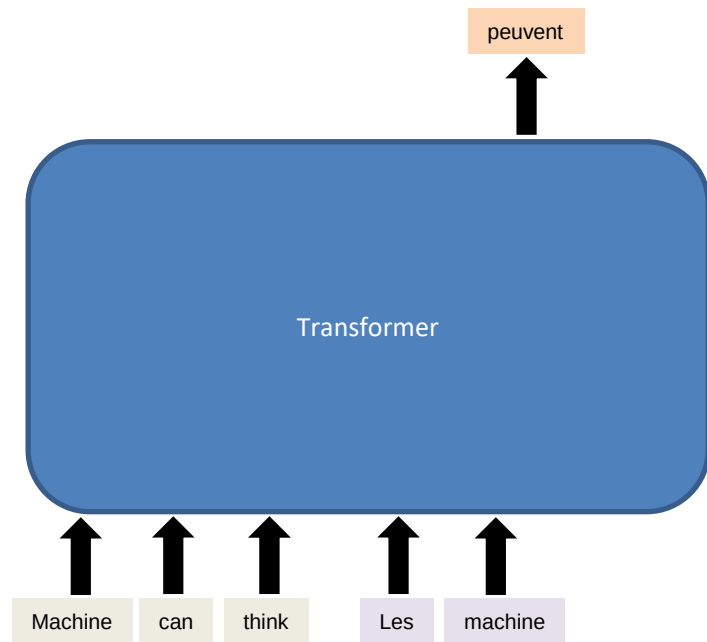
Attention mechanism in RNN



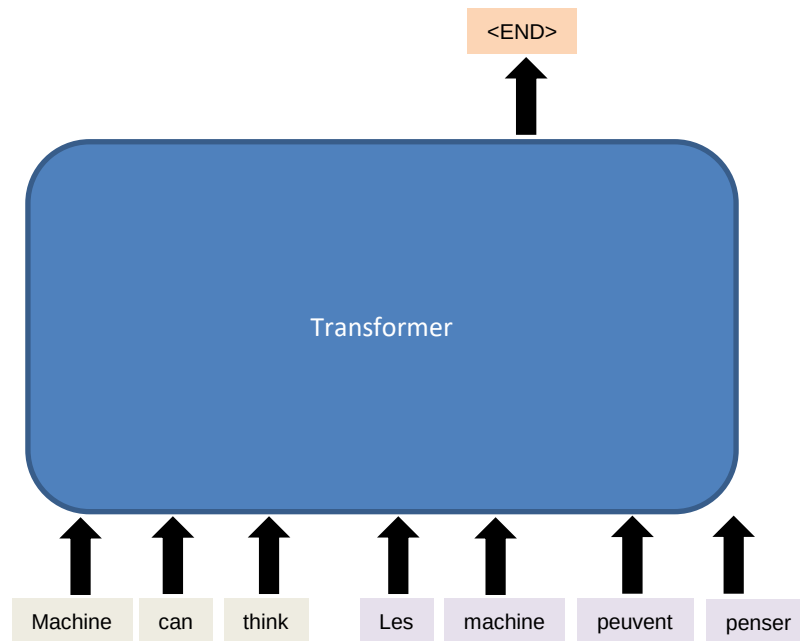
Transformers



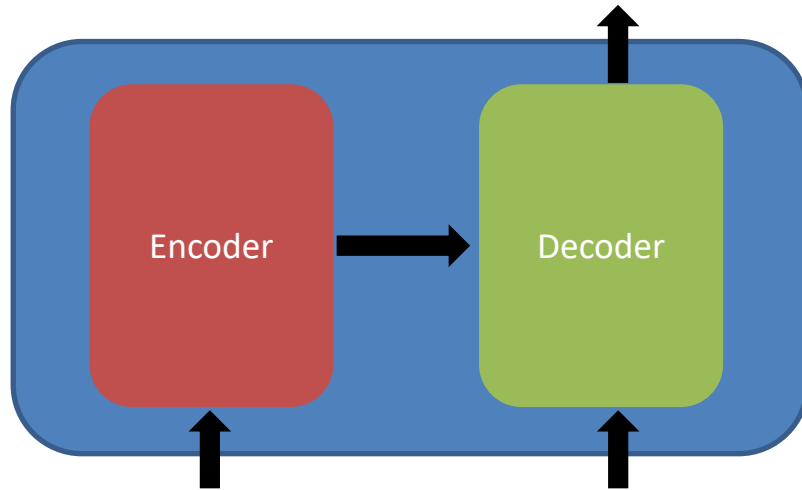
Transformers



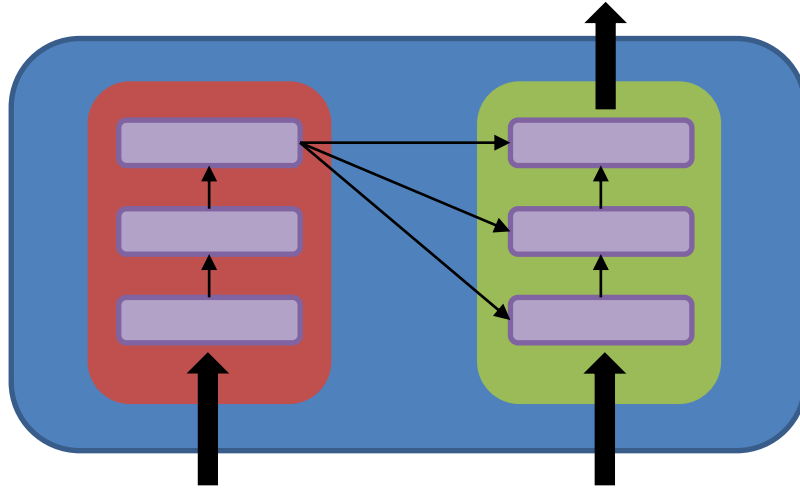
Transformers



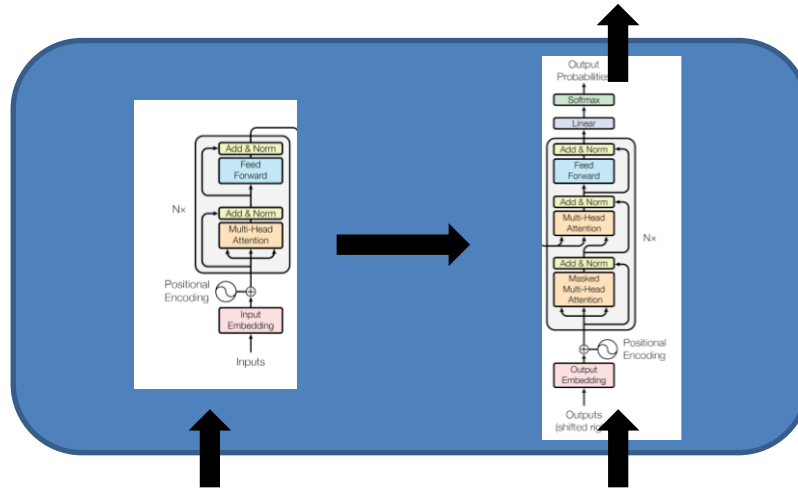
Transformers



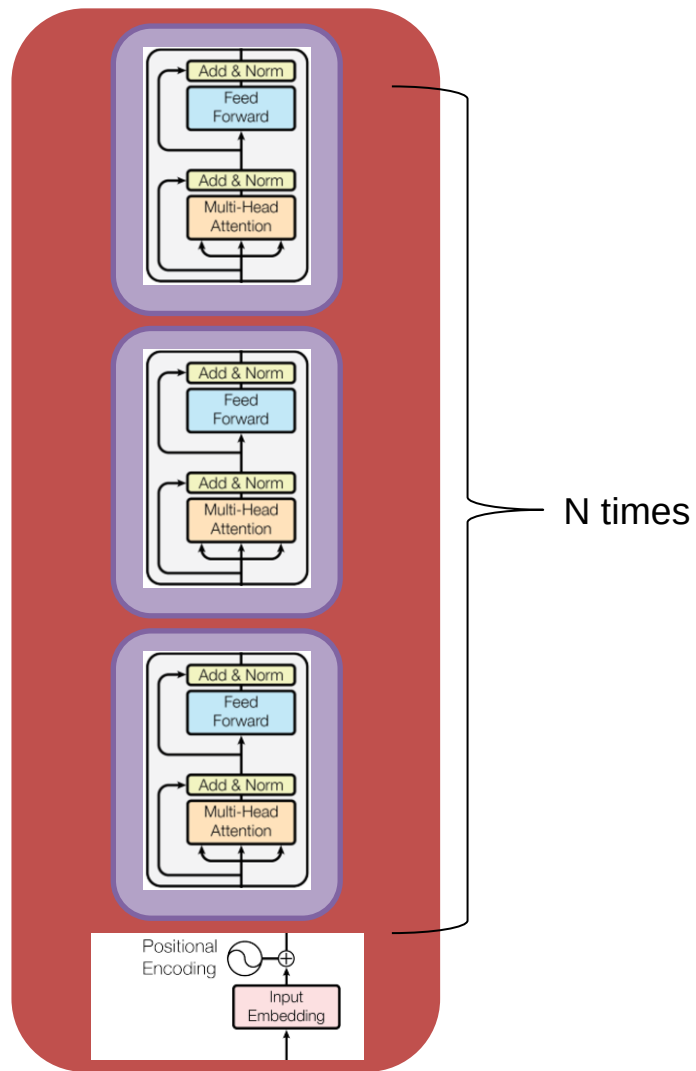
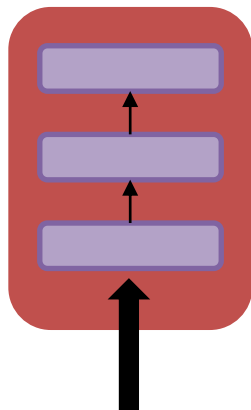
Transformers



Transformers



Encoder



Question for you?

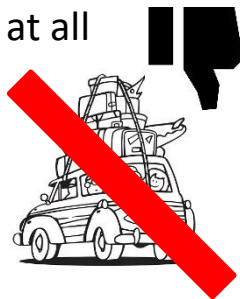
- Now that transformer architecture has removed the need to have RNN architectures, how do the transformers keep information about the order of the words?



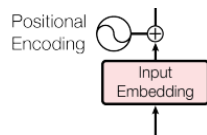
The trip was **good**, not **bad** at all

vs.

The trip was **bad**, not **good** at all



Encoder – input embedding & positional encoding

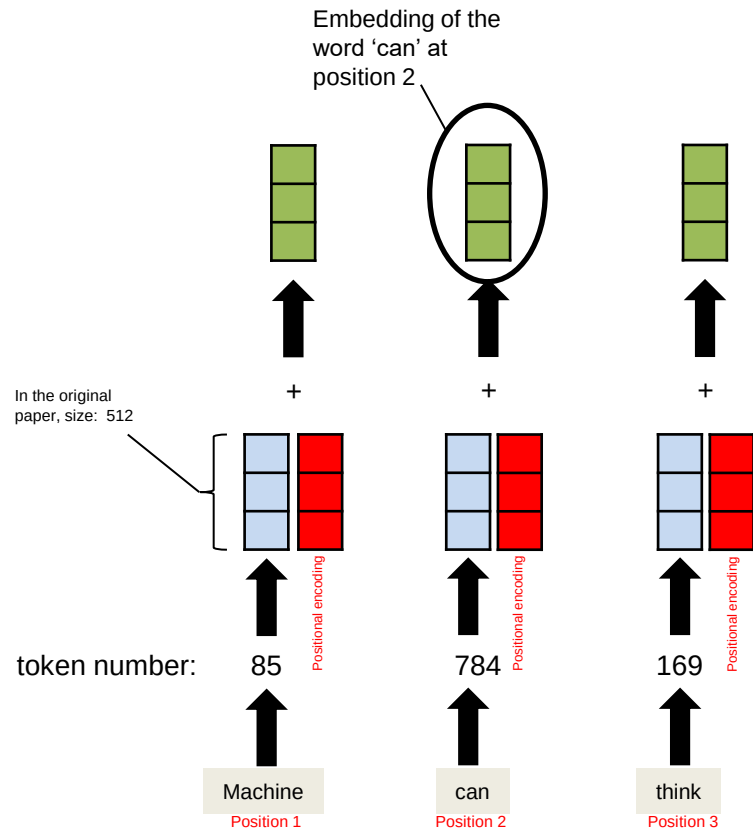


The input embedding in a transformer's encoder converts discrete input tokens (e.g., words) into continuous vectors. These embeddings capture semantic information and enable the model to process the input sequence.

Positional encoding is a technique that adds information about a token's position in a sequence to its embedding. It helps the transformer model understand the order of tokens, enabling it to process and learn from sequences effectively.

Positional encoding vectors, generated for each position in a sequence, are element-wise added to their corresponding input embeddings. This combination retains token and position information, serving as input for the transformer's subsequent layers.

Note that in the "Attention is All You Need" paper, the authors proposed a specific positional encoding using sine and cosine functions with different frequencies.



The positional encoding used in the paper "Attention is All You Need"

- The position encoding can be learned by the model, but the authors of the original paper have proposed an analytical way to compute this positional encoding.
- This encoding strategy is still widely used in many modern models.

- $$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

- $$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

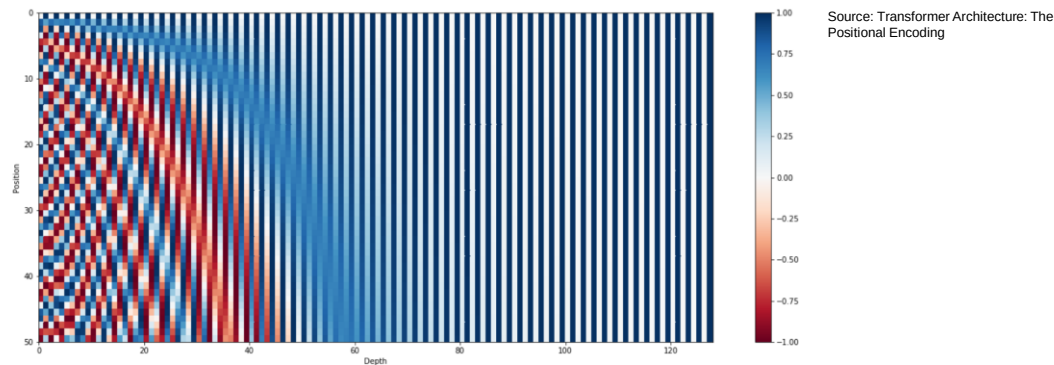
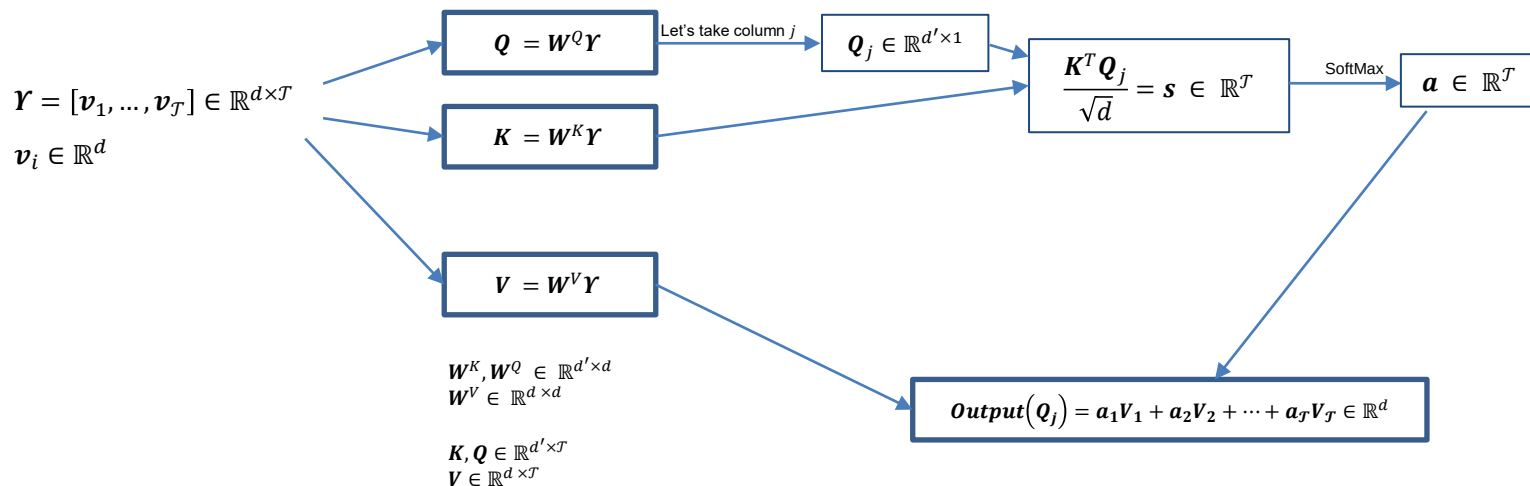
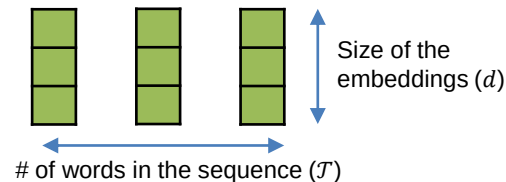


Figure 2 - The 128-dimensional positional encoding for a sentence with the maximum length of 50. Each row represents the embedding vector $\vec{p}_i$

- For more info:
https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

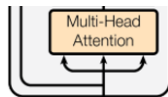
Self attention

- Before going to the Multi-head (self) attention, let's have a look at how **one-head (self) attention** works.
- The input of the attention mechanism is a vector of vectors (i.e., yes, a matrix ;-))

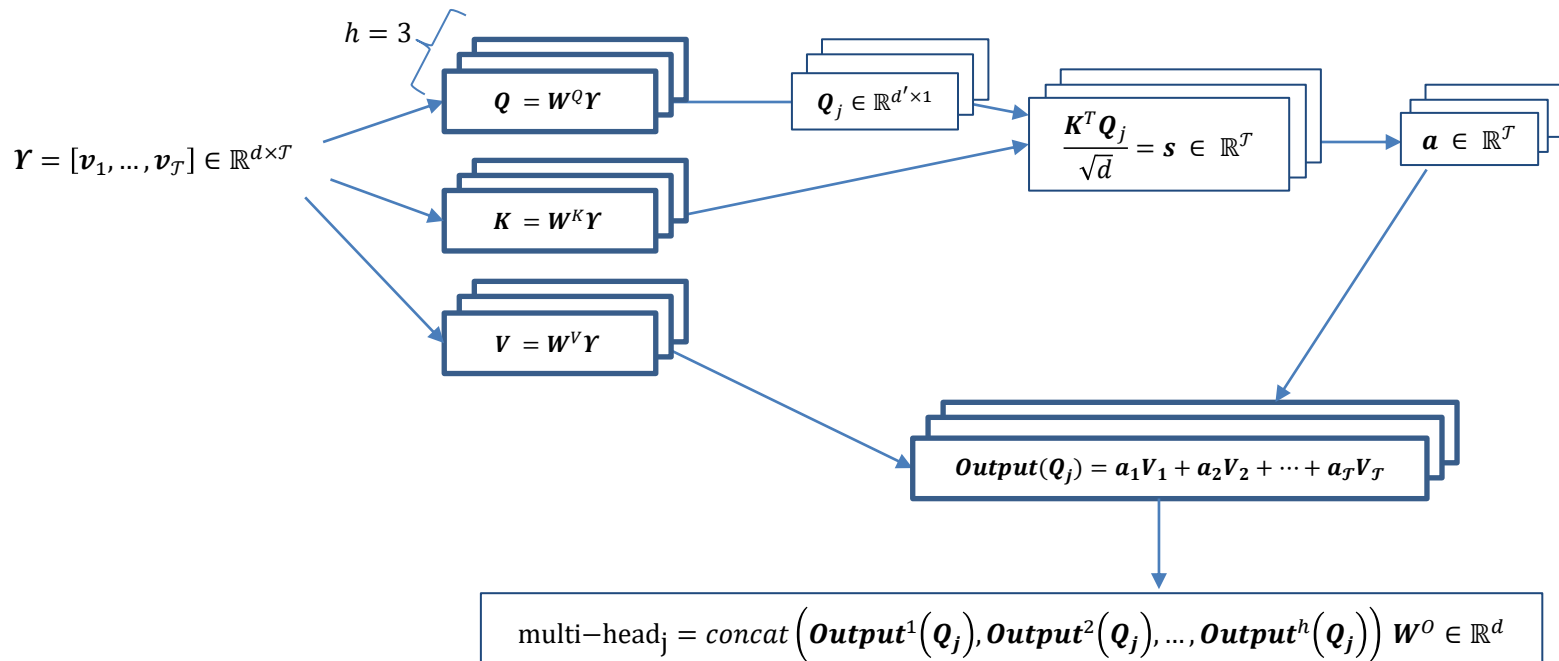


This is repeated for all word $j \in [1, \dots, T]$

Multi-head (self) attention



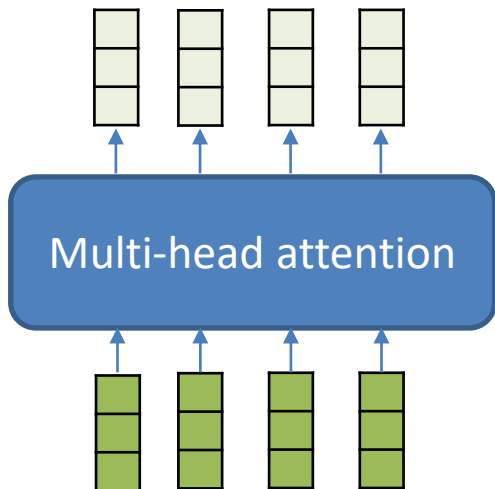
- Self-attention calculates input sequence relationships.
- Multi-head self-attention does this in parallel with different weight matrices W^K , W^Q and W^V .



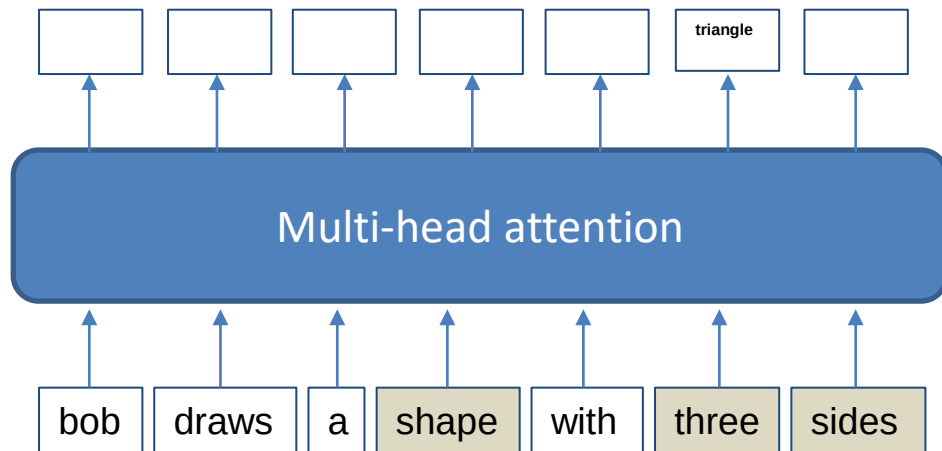
This is repeated for all word $j \in [1, \dots, T]$

Multi-head (self) attention

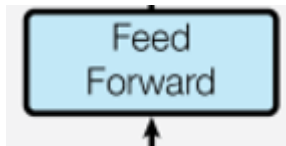
The multi-head attention function receives T (here $T = 4$) vectors of dimension d (here $d = 3$) as input and returns the same number of vectors of the same dimension, but the new vectors should have a better representation of the information.



The new vectors should have a better representation of the information.

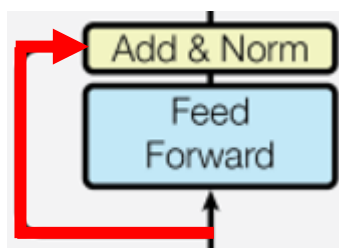
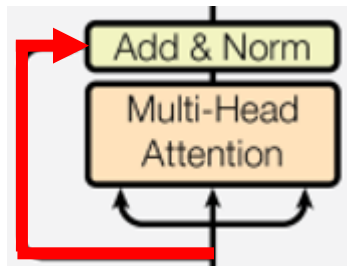


Feed Forward



- It is used to process the output of the multi-head attention layer
- It consists of two linear transformations with a ReLU activation in between.
- **The Feed Forward module is applied independently to each position in the sequence!!**
- In original Transformer, it is composed of two linear transformations with a ReLU activation in between
$$\mathbf{W}_2 \text{ ReLu}(\mathbf{W}_1 \mathbf{x} + b_1) + b_2$$
- The input of the Feed Forward module is first passed through a linear layer with a largely hidden dimension, followed by a ReLU activation. This is then followed by another linear layer that projects the activations back to the original dimensionality.

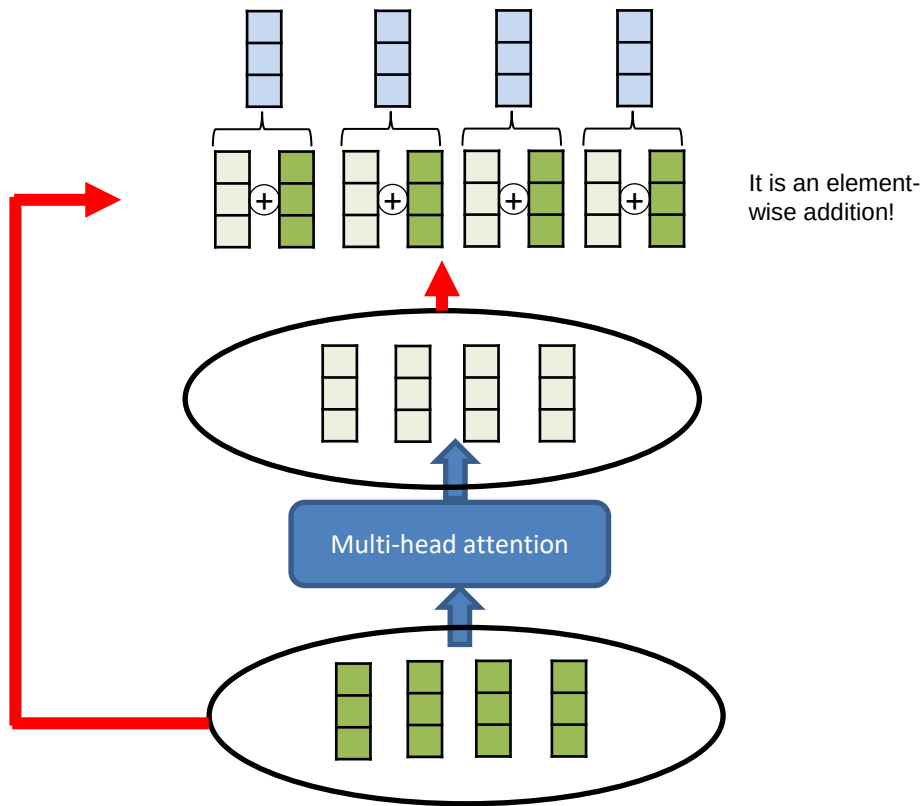
Add



After the Multi-head attention and Feed Forward, we see that the signal is bypassed.

Question for you?

- Why do we do this?



In each multi-head attention or feedforward layer of the Transformer, the output of the attention or feedforward sub-layer is added to the input of that sub-layer before being passed on to the next sub-layer.

Layer Normalization



Warning! This is not Batch normalization

Reminder: Batch normalization

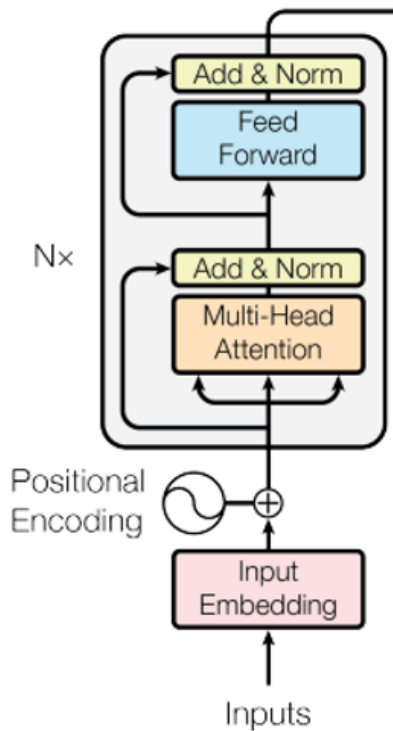
- It is used to improve training stability and speed up convergence.
- It involves normalizing the activations of a given layer by subtracting the mean and dividing by the standard deviation of the batch of inputs.
- This normalization is done per neuron separately, i.e., each neuron has its own mean and standard deviation values computed over the batch.
- Batch normalization is typically applied before the activation function of a layer (i.e, the z).
- γ and β are learnable parameters.
- It requires storing a running estimate of the mean and standard deviation values during training.

Question for you: Why is it needed to make these running estimates?

Layer Normalization

- This normalization operation used in Transformers is called *Layer Normalization*.
- It is applied to the activations across the feature dimension of a single layer (rather than across batches!!!) on a single training/testing case.
- For a given layer, each element in the feature dimension is normalized using the mean and standard deviation computed across all elements in that same feature dimension.
- It also has learnable parameters to do scaling and shifting.

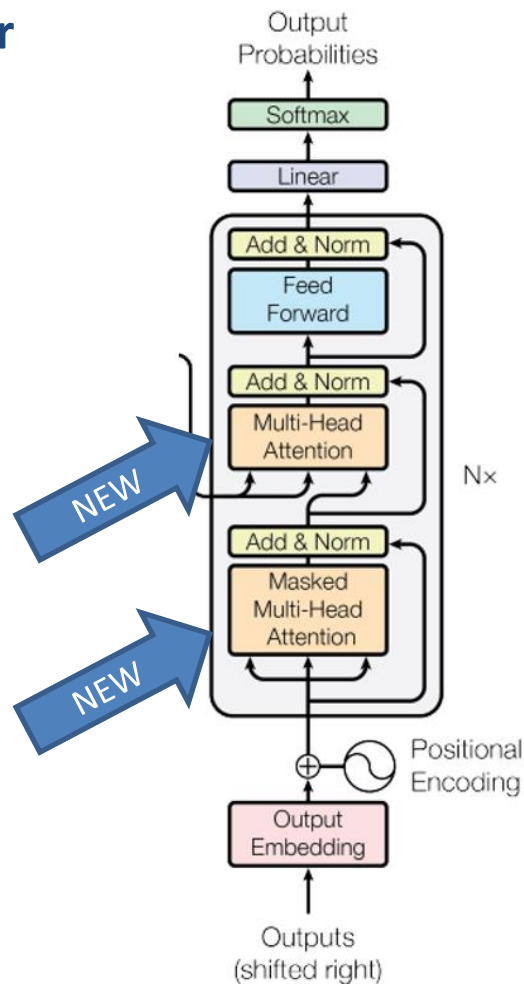
Transformer – Encoder



Question for you:

- Can you explain each piece of this figure?
- If the purpose is to make an English-to-French translator
 - What is the input of the encoder?
 - What is the output of the encoder?

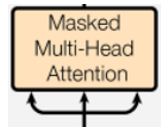
Transformer – Decoder



Question for you:

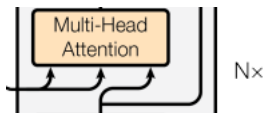
- Can you explain each of the 'not-new' pieces of this figure?
- If the purpose is to make an English-to-French translator
 - What is the input of the decoder?
 - What is the output of the decoder?

Masked Multi-head (self) attention

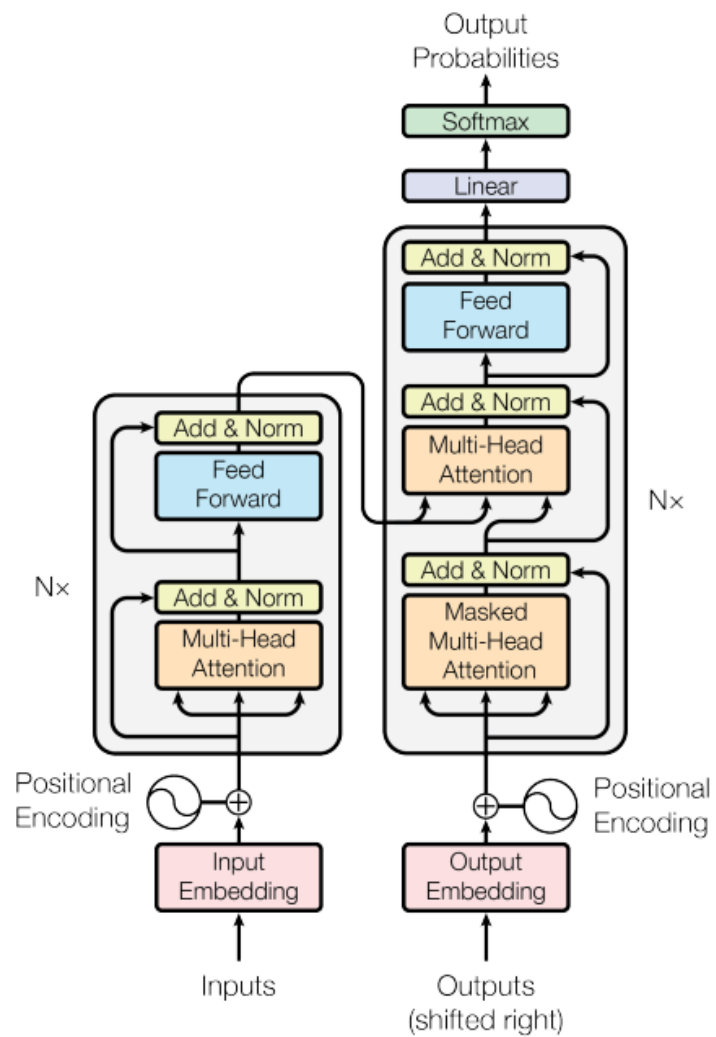


- The masked multi-head (self) attention is in the decoder.
- When the model must predict the next word, it does not have access to the future.
- It prevents the model from attending to future tokens during training.
- The masked multi-head self-attention module is an important part of the Transformer architecture, and it has been shown to be effective for many natural language processing tasks.

Multi-head (cross) attention



- If keys, values, and queries are generated from the same sequence, **it is called self-attention**.
- **In cross-attention**, the queries are generated by a **different** sequence, than the key-value pairs.



Learning an LLM

Pretraining



- Our mode is very good for next Token Prediction.
- This training was done on raw text from internet

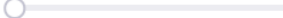


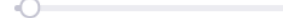
Please can you suggest me a sports program for the day? 

I am a beginner and I want to start with a sports program.

Mode
☰ Complete ▼

Model
davinci ▼

Temperature 0


Maximum length 118


Something is wrong with this model!

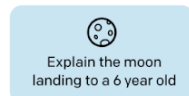


From GPT to InstructGPT (Reinforcement Learning with Human Feedback (RLHF))

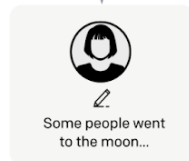
Step 1

Collect demonstration data, and train a supervised policy.

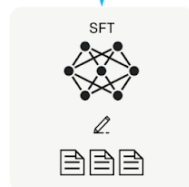
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



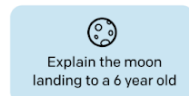
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

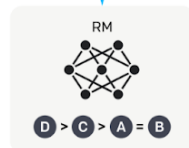
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.

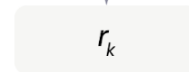


Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



From GPT to InstructGPT (Reinforcement Learning with Human Feedback (RLHF))

- **Supervised Fine-Tuning (SFT) Stage:**

- In this initial phase, human labelers provide responses to a range of random prompts, effectively creating a supervised training set.
- The model is then fine-tuned on this dataset, developing an initial understanding of the instructions and desired output, thereby creating an SFT model.
- This SFT stage is crucial in allowing the model to understand and learn the expected behavior pattern, that is, producing responses as close as possible to the 'ideal' ones given by the human labelers.

- **Reward Model (RM) Construction:**

- To create a reward model, multiple responses to the same prompt generated by the SFT model are ranked by human labelers based on their quality and correctness.
- These rankings quantify the quality of a response and serve as a basis to guide the model towards more ideal responses in future iterations.

Step 1

Collect demonstration data, and train a supervised policy.

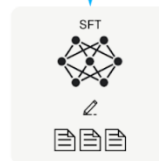
A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

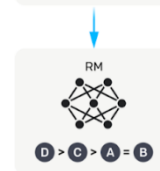
Explain the moon landing to a 6 year old

A B
Explain gravity... Explain war...
C D
Moon is natural satellite of... People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.



From GPT to InstructGPT (Reinforcement Learning with Human Feedback (RLHF))

Reinforcement Learning Process

- At this stage, we start with a GPT-3 model that has already been fine-tuned using Supervised Fine-Tuning (SFT), which means it has been trained on a dataset of prompts and corresponding ideal responses provided by human labelers.
- This fine-tuned model generates responses to a new set of prompts, which are subsequently assessed and ranked by the Reward Model (RM).
- The feedback or 'rewards' from the RM are used to guide the reinforcement learning process. Specifically, a machine learning technique called Proximal Policy Optimization (PPO) is used to fine-tune the SFT model further, aiming to improve the quality of its responses based on the feedback from the RM.
- This process of generating responses, evaluating them with the RM, and updating the model with PPO is iteratively repeated.
- After numerous iterations, the end result is an enhanced model that is more adept at following instructions and generating high-quality responses, aptly named as the 'Instructed GPT-3' model.

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.

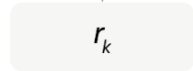


Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Please can you suggest me a sports program for the day?

- 1. Warm-up: 10 minutes of light jogging, followed by dynamic stretching
- 2. Strength Training: 30 minutes of bodyweight exercises such as squats, lunges, push-ups, and planks
- 3. Cardio: 20 minutes of interval training on a treadmill or elliptical
- 4. Core Work: 10 minutes of core exercises such as crunches, Russian twists, and leg raises

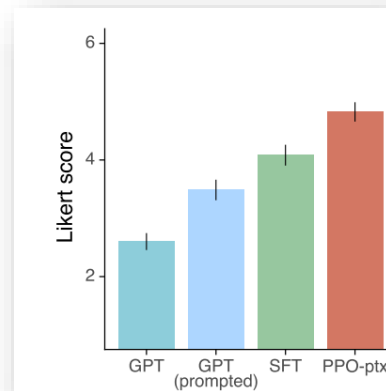
Mode: Complete

Model: text-davinci-003

Temperature: 0

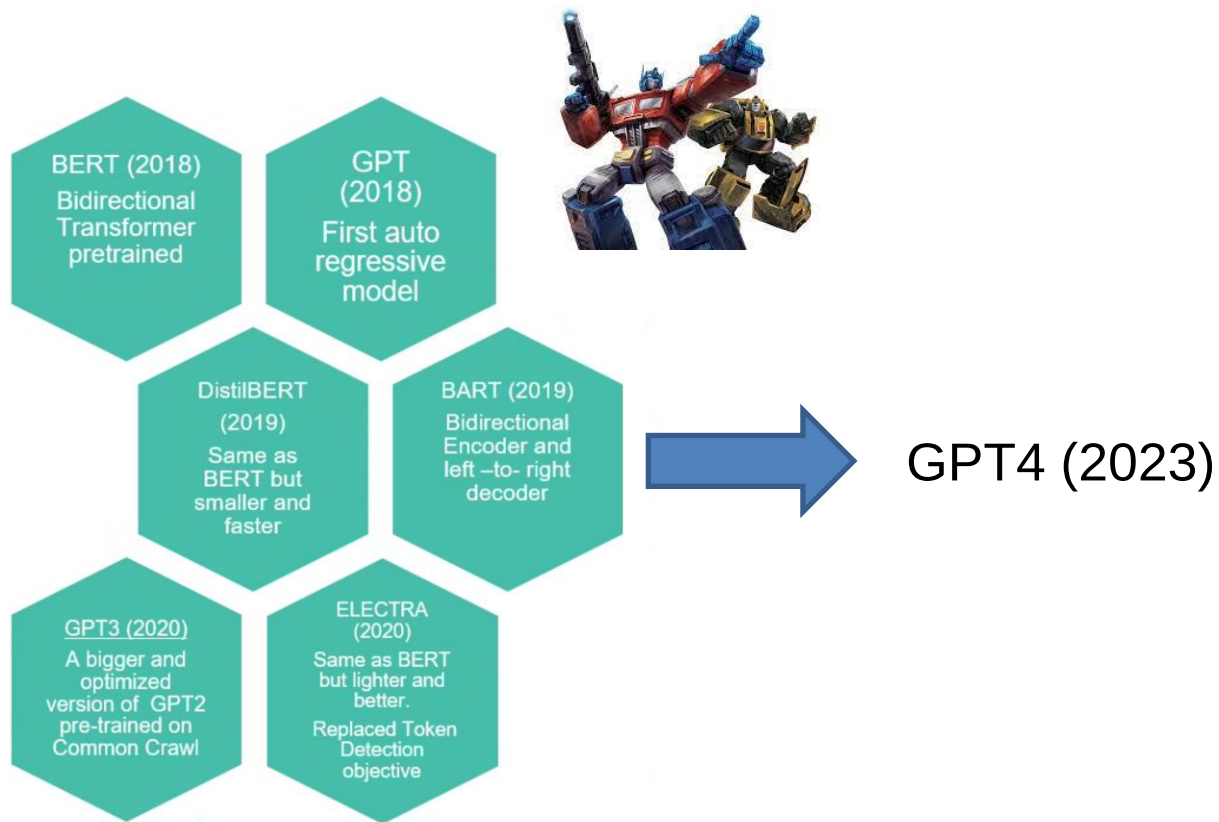
Maximum length: 118

It's better!!



Question for you: This model is 'text-davinci-003' which is not designed to follow a chat conversation. How do you think they managed to enable this model to follow a chat-like conversation?

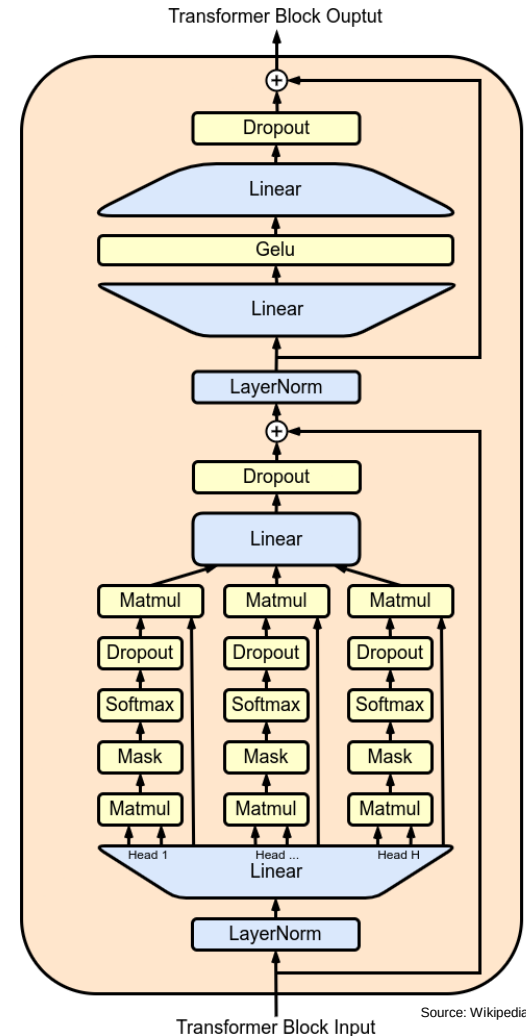
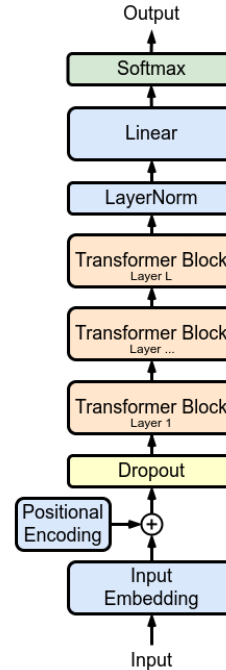
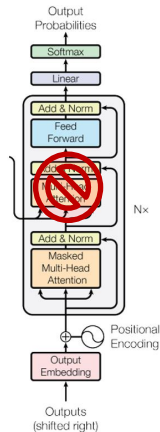
Transformers families (in NLP)



GPT-1

- The original transformer was designed to perform sequence-to-sequence tasks, whereas GPT-1 was developed for language modeling tasks.
- GPT-1 uses a decoder-only transformer architecture, whereas the original transformer uses an encoder-decoder architecture.
- Additionally, GPT-1 uses masked self-attention, whereas the original transformer uses both masked and unmasked self-attention.
- GPT-1 has 117 million parameters.
- GPT-1 uses the dataset BooksCorpus to pre-train the language model

A language model like GPT is similar to the decoder of the transformer without cross-attention



GPT-2

- GPT-2 uses a much larger dataset (WebText) compared to the BooksCorpus dataset used in GPT-1
- GPT-2 has significantly more parameters, with 1.5 billion compared to GPT-1's 117 million.
- GPT-2 uses a larger context window than GPT-1.

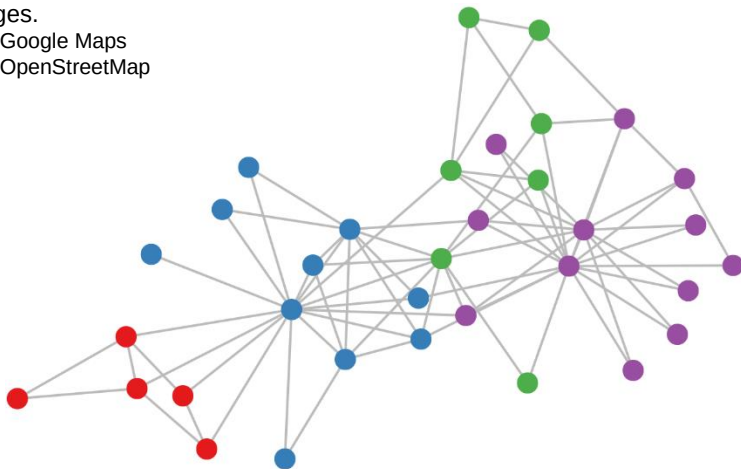
GPT-3

- GPT-3 has 175 billion parameters, which is much larger than GPT-2's 1.5 billion parameters.
- GPT-3 was trained on a mix of five different corpora, while GPT-2 was trained on the WebText dataset.
- The word embeddings in GPT-3 are larger than those in GPT-2.
- GPT-3 has a larger context window size than GPT-2.

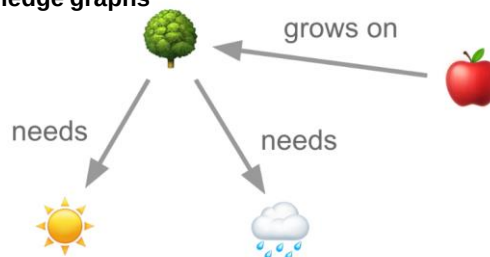
Graph neural network (GNN)

Graph-structured data

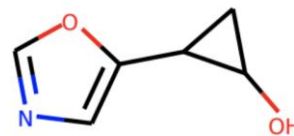
- **Social networks:** Representing people as nodes and their relationships (friends, followers, etc.) as edges.
 - Facebook Network
 - Twitter (X) Follow Graph
 - LinkedIn Connections
- **Citation networks:** Nodes represent academic papers, and edges represent citations between them.
 - PubMed
 - arXiv
 - Google Scholar Citations
- **World Wide Web:** Web pages are nodes, and hyperlinks are edges.
 - Web Link Graph
 - Site Interconnectivity
- **Road networks:** Intersections and endpoints are nodes, and roads are edges.
 - Google Maps
 - OpenStreetMap
- ...



Knowledge graphs



Molecule structures
Proteins
Physical systems
...



How can we effectively develop ML models for this kind of data?

Graph - formalization

- Collection of entities (**nodes**)
- Pairwise interaction/relation (**edges**)

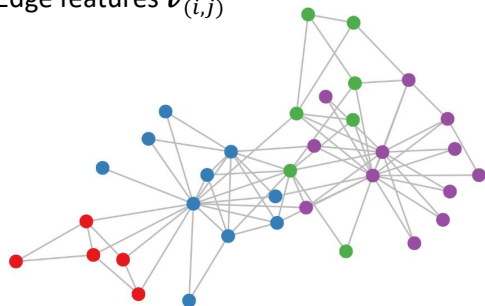
Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$

Nodes $i \in \mathcal{V}$

Edges $(i, j) \in \mathcal{E}$

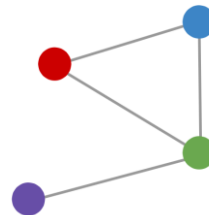
None features $\mathbf{v}_i$

Edge features $\mathbf{v}_{(i,j)}$



Adjacency Matrix A

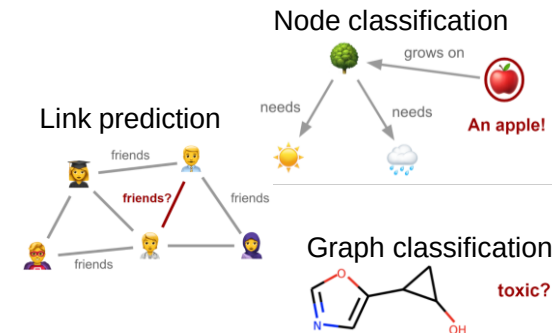
				
	0	1	1	0
	1	0	1	1
	1	1	0	0
	0	1	0	0



Typical variants:

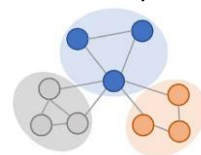
- Directed graph
- Weighted graph
- Self-connection ($A + I$)
- Multi-relation graph (e.g.; knowledge graphs)
Edges $(i, r, j) \in \mathcal{E}$ and relation type $r \in \mathcal{R}$.

Typical supervised tasks on graphs

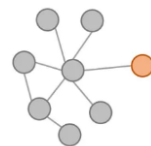


Typical unsupervised tasks on graphs

Community detection

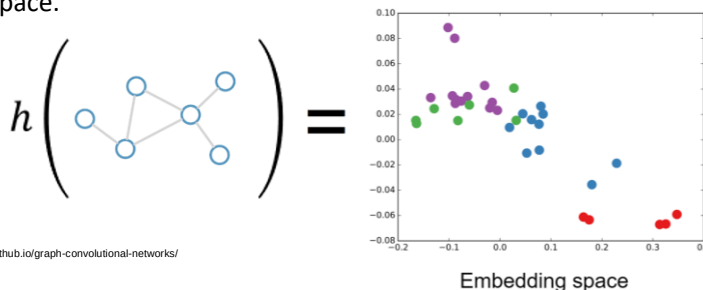


Anomaly detection



Graph representation learning (embedding)

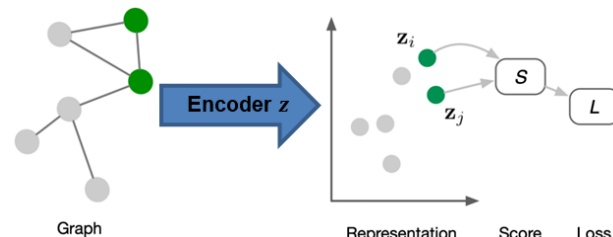
- Graph representation learning aims to **encode** the information contained in a graph into a **low-dimensional** space.



- This embedding represents the nodes, edges, or entire graphs in a way that **preserves the structural and feature information**.
- This encoding facilitates tasks like:
 - clustering
 - classification
 - link prediction
 -

Embedding learning: discussion

- Cannot generalize the new nodes at test time (with unseen IDs)
- No parameter sharing between nodes (every node is embedded using a separated learned vector)



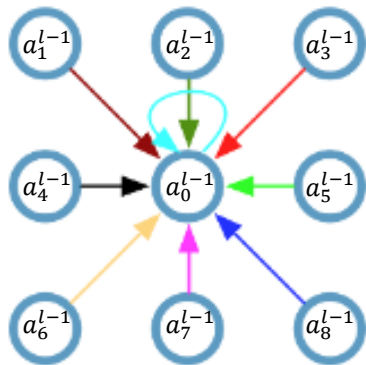
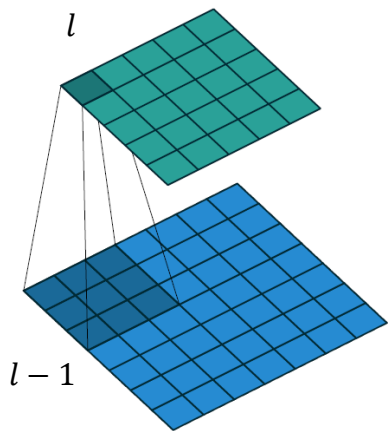
Method	Score $s(i, j)$	Loss
Laplacian Eigenmaps	$\ z_i - z_j\ _2^2$	$\sum_{i,j} s(i, j) A_{ij}$
node2vec	$P(j i) = \frac{\exp(z_i^T z_j)}{\sum_k \exp(z_i^T z_k)}$	$-\sum_{i \in V} \log P(N_S(i) i)$

Search the function z that minimizes "Loss"

node2vec

- Random Walks Generation:** Start random walks from each node in the graph. The walks can be biased to explore different structures in the graph.
- Mapping Walks to Sentences:** Treat each random walk as a sentence, where the nodes are analogous to words.
- Word Embedding:** Use word2vec techniques to learn embeddings for the nodes.
- Vector Representations:** The learned embeddings represent the nodes in a continuous vector space, where the geometry of the space encodes the structural properties of the nodes in the graph.

From convolution neural network to convolution-based graph neural networks



Note that: if multi-channels, $\mathbf{a}_j^{l-1}$
and the $\mathbf{w}$'s are vectors



Convolution neural network

$$\mathbf{a}_0^l = f^{l-1} \left(\sum_{j=0}^8 \mathbf{w}_j^T \mathbf{a}_j^{l-1} \right)$$

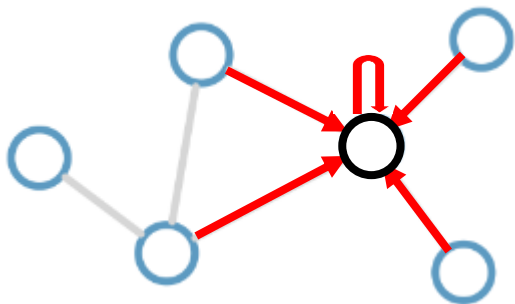


Convolution-based graph neural network

$$\mathbf{a}_0^l = f^{l-1} \left(\mathbf{w}_0^T \mathbf{a}_0^{l-1} + \sum_{j=1}^8 \mathbf{w}_1^T \mathbf{a}_j^{l-1} \right)$$

Except for the fact that the same parameter $\mathbf{w}_1$ is reused for all pixel neighbors, GCN and CNN are very similar at first glance.

Convolution-based graph neural networks



Note: In this equation, the bias term is omitted but it should be there.

$$a_i^l = f^{l-1} \left(\mathbf{w}_0^T \mathbf{a}_i^{l-1} + \sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{w}_1^T \mathbf{a}_j^{l-1} \right)$$

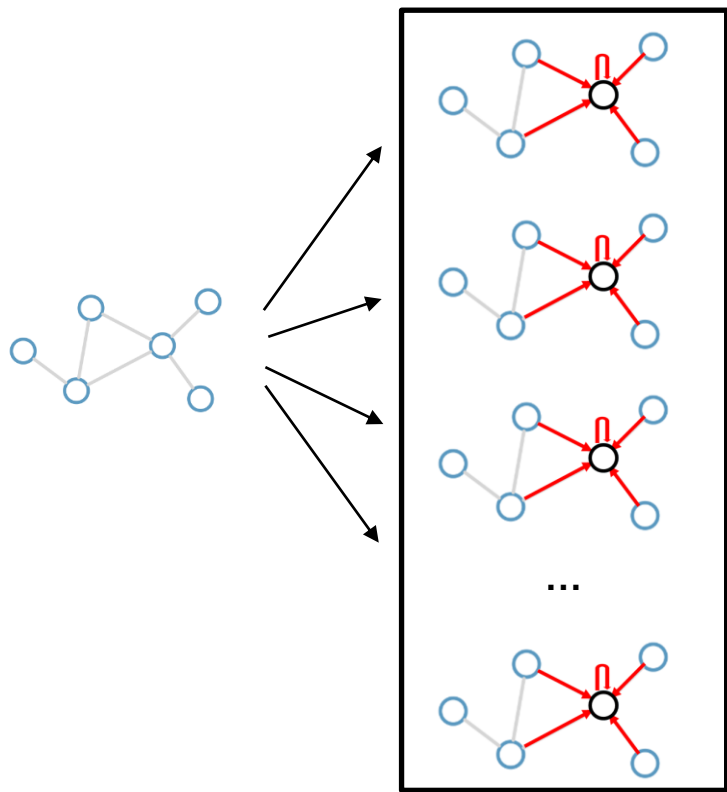
Neighbor indices of i (Optional) normalization term

- Each node at level $l - 1$ is represented by a vector of dimension n_{l-1} :
 $\mathbf{a}_i^{l-1} \in \mathbb{R}^{n_{l-1} \times 1}$
- The two vectors $\mathbf{w}_0$ and $\mathbf{w}_1$ have the same dimension than $\mathbf{a}_i^{l-1}$:
 $\mathbf{w}_0 \in \mathbb{R}^{n_{l-1} \times 1}$ and $\mathbf{w}_1 \in \mathbb{R}^{n_{l-1} \times 1}$
- For each node i , the normalization term sum to 1 :
 $\forall i \in \mathcal{V}, \sum_j \alpha_{ij} = 1$

Remarks:

- Instead of sum, we can use other aggregation functions (e.g. max).
- f can be any activation function... but can also be a neural net itself!

Convolution-based graph neural networks



As with **multilayer perceptron**, where it is possible to have **many neurons per layer**, and as with **CNN**, where it is possible to have **many channels**, in a **GCN** it is also possible to have **many convolutions per layer**.

$$\mathbf{a}_i^l = f^{l-1} \left(\mathbf{W}_0 \mathbf{a}_i^{l-1} + \sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W}_1 \mathbf{a}_j^{l-1} \right) \in \mathbb{R}^{n_l}$$

where : $\mathbf{W}_0 \in \mathbb{R}^{n_l \times n_{l-1}}$ and $\mathbf{W}_1 \in \mathbb{R}^{n_l \times n_{l-1}}$

A note on matrix representation

Instead of

$$\mathbf{a}_i^l = f^{l-1} \left(\mathbf{w}_0 \mathbf{a}_i^{l-1} + \sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{w}_1 \mathbf{a}_j^{l-1} \right) \in \mathbb{R}^{n_l}$$

You will often see a matrix representation of the update rule:

$$\mathbf{A}^l = f^{l-1} (\mathbf{W}_0 \mathbf{A}^{l-1} + \mathbf{W}_1 \mathbf{A}^{l-1} \mathcal{A}) \in \mathbb{R}^{n_l \times |\mathcal{V}|}$$

Where :

$$\mathbf{A}^{l-1} \in \mathbb{R}^{n_{l-1} \times |\mathcal{V}|}$$

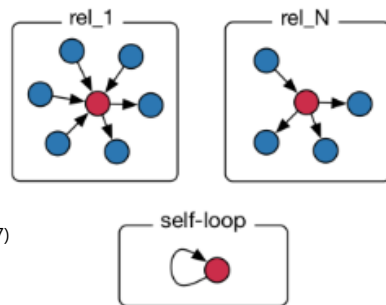
$$\mathcal{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|} \quad \text{where the column of } \mathcal{A} \text{ sums to 1}$$

More expressive GNN variants

- Adding support for relational types

$$\mathbf{a}_i^l = f^{l-1} \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i} \alpha_{ij}^r \mathbf{W}_{1,r} \mathbf{a}_j^{l-1} \right) \in \mathbb{R}^{n_l}$$

Paper: *Modeling Relational Data with Graph Convolutional Networks*, Schlichtkrull (2017)

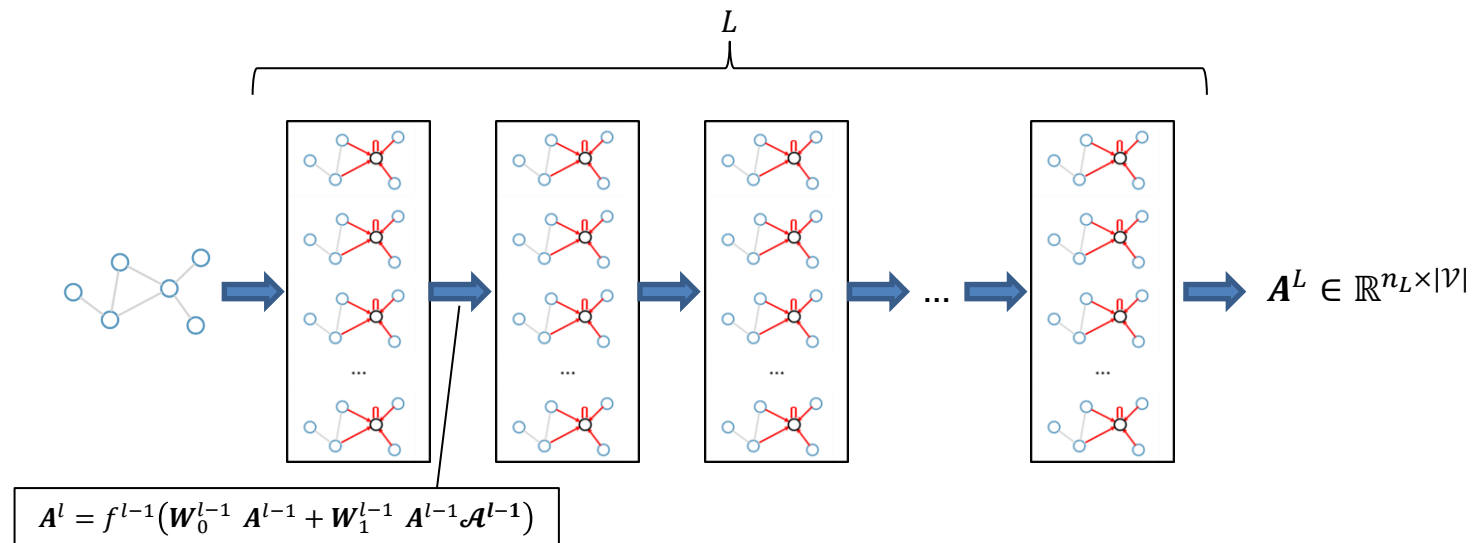


- Attention based aggregation

$$\alpha_{ij}^r = \frac{\left((\mathbf{a}_i^{l-1})^T \mathbf{W}'_{1,r} \mathbf{a}_j^{l-1} \right)}{\sum_{k \in \mathcal{N}_i} \left((\mathbf{a}_i^{l-1})^T \mathbf{W}'_{1,r} \mathbf{a}_k^{l-1} \right)}$$

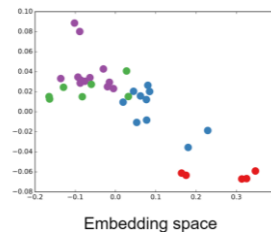
Paper: *Do Transformers Really Perform Bad for Graph Representation?*, Ying (2021)

Layers



$|\mathcal{V}|$ vectors: one for each node

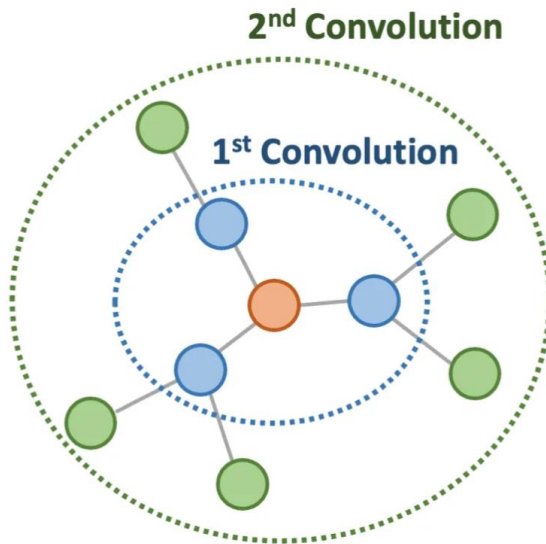
$$h\left(\begin{array}{c} \text{graph} \end{array}\right) = A^L = \begin{pmatrix} \vdots & \vdots & \vdots \end{pmatrix}$$



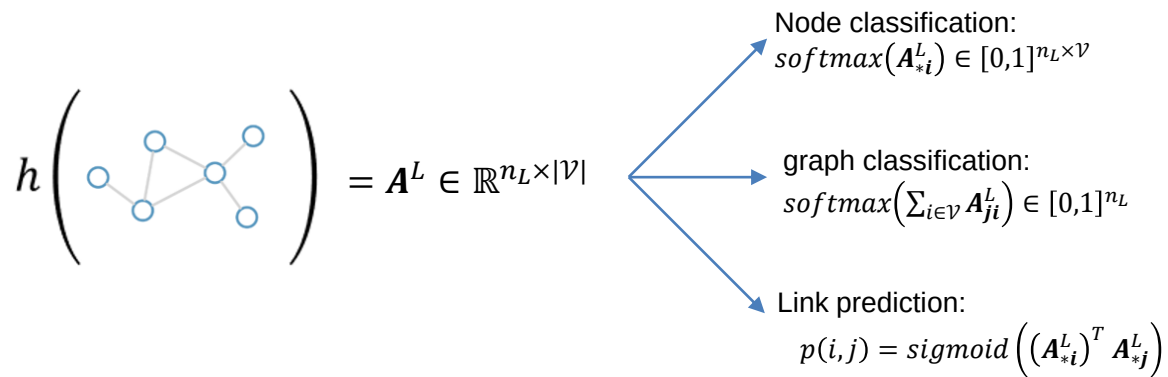
All of the $|\mathcal{V}|$ nodes are now represented in a space of n_L dimension (e.g. $n_L = 2$)

Layers

- The number of graph convolutions determines how many steps away node features are aggregated into each node.
- In the image below, the first convolution aggregates the blue nodes' features into the orange node and the second convolution can incorporate the green nodes' features into the orange node.



One fit all: Classification and link prediction with GNNs



Torch Geometric (PyG)



- A single graph in PyG is described by an instance of “`torch_geometric.data.Data`”.

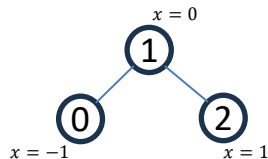
```
import torch
from torch_geometric.data import Data

edge_index = torch.tensor([[0, 1, 1, 2],
                           [1, 0, 2, 1]], dtype=torch.long)
x = torch.tensor([[-1], [0], [1]], dtype=torch.float)

data = Data(x=x, edge_index=edge_index)

print(data)
```

✓ 0.0s



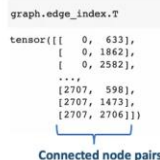
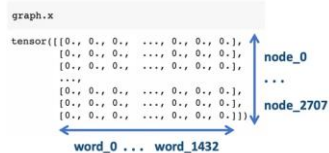
Although the graph has only two edges, we need to define four index tuples to account for both directions of an edge.

- The **Cora** dataset:
 - 2708 scientific publications classified into one of seven classes
 - The citation network consists of 5429 links
 - Each publication is described by a binary vector indicating the presence of a corresponding word
 - the dictionary consists of 1433 unique words.

```
from torch_geometric.datasets import Planetoid

dataset = Planetoid(root='/tmp/Cora', name='Cora')
graph = dataset[0]
```

✓ 4.0s



Each node has one of seven classes, which will be the model target:

```
"""
Class Definition
0: Theory
1: Reinforcement_Learning
2: Genetic_Algorithms
3: Neural_Networks
4: Probabilistic_Methods
5: Case_Based
6: Rule_Learning
"""
```

```
pd.DataFrame(graph.y)[0].value_counts()
✓ 0.0s
```

0	
3	818
4	426
2	418
0	351
5	298
1	217
6	180

It is like a dictionary

```
print(data.keys)
print(data['edge_index'])
print(data['x']) # node feature
✓ 0.0s
```

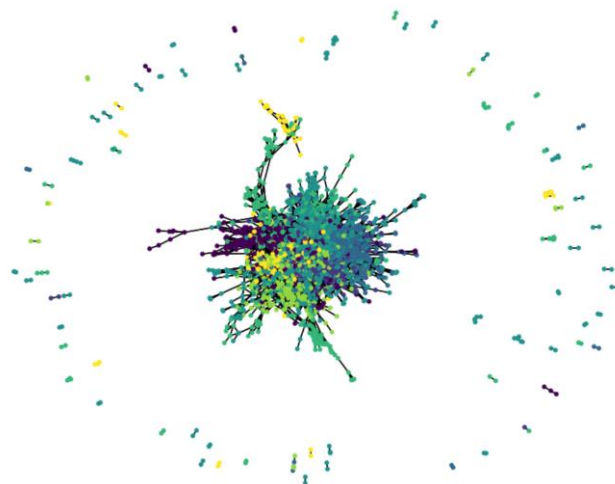
['edge_index', 'x']

```
tensor([[0, 1, 1, 2],
        [1, 0, 2, 1]])
tensor([[-1.],
        [ 0.],
        [ 1.]])
```

With some additional graph features

```
print(f"{data.num_nodes=}")
print(f"{data.num_edges=}")
print(f"{data.num_node_features=}")
print(f"{data.has_isolated_nodes()}")
print(f"{data.has_self_loops()}")
print(f"{data.is_directed()}")
✓ 0.0s
```

```
data.num_nodes=3
data.num_edges=4
data.num_node_features=1
data.has_isolated_nodes()=False
data.has_self_loops()=False
data.is_directed()=False
```



Node classification on Cora

- First, we must split the nodes between train, validation & test.

```
import torch_geometric.transforms as T

split = T.RandomNodeSplit(num_val=0.1, num_test=0.2)
graph = split(graph)

✓ 0.0s
```

The data splits are written into mask attributes in the graph object (see the code below) instead of splitting the graph itself.

```
print(graph)
print(graph['train_mask'])
print(graph['val_mask'])
print(graph['test_mask'])

✓ 0.0s

Data(x=[2708, 1433], edge_index=[2, 10556], y=[2708], train_mask=[2708], val_mask=[2708], test_mask=[2708])
tensor([ True, False,  True, ...,  True,  True, False])
tensor([False, False, False, ..., False, False,  True])
tensor([False,  True, False, ..., False, False, False])
```

- First, we will compute a baseline with a simple multi-layer perceptron only using node features.
 - The model ignores the graph structure and tries to classify the node labels only using the word vectors.
- Next, we are training GCN and comparing its performance to MLP.

Baseline Model (MLP) on Node Classification

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(dataset.num_node_features, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, dataset.num_classes)
        )

    def forward(self, data):
        x = data.x # only using node features (x)
        output = self.layers(x)
        return output
```

✓ 0.0s

```
def train_node_classifier(model, graph, optimizer, criterion, n_epochs=200):

    for epoch in range(1, n_epochs + 1):
        model.train()
        optimizer.zero_grad()
        out = model(graph)
        loss = criterion(out[graph.train_mask], graph.y[graph.train_mask])
        loss.backward()
        optimizer.step()

        pred = out.argmax(dim=1)
        acc = eval_node_classifier(model, graph, graph.val_mask)

        if epoch % 20 == 0:
            print(f'Epoch: {epoch:03d}, Train Loss: {loss:.3f}, Val Acc: {acc:.3f}')

    return model
```

```
def eval_node_classifier(model, graph, mask):

    model.eval()
    pred = model(graph).argmax(dim=1)
    correct = (pred[mask] == graph.y[mask]).sum()
    acc = int(correct) / int(mask.sum())

    return acc
```

✓ 0.0s

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
mlp = MLP().to(device)
graph = graph.to(device)

optimizer_mlp = torch.optim.Adam(mlp.parameters(), lr=0.01, weight_decay=5e-4)
criterion = nn.CrossEntropyLoss()
mlp = train_node_classifier(mlp, graph, optimizer_mlp, criterion, n_epochs=200)

test_acc = eval_node_classifier(mlp, graph, graph.test_mask)
print(f'Test Acc: {test_acc:.3f}')
```

✓ 2.8s

```
Epoch: 020, Train Loss: 0.068, Val Acc: 0.764
Epoch: 040, Train Loss: 0.014, Val Acc: 0.742
Epoch: 060, Train Loss: 0.014, Val Acc: 0.731
Epoch: 080, Train Loss: 0.011, Val Acc: 0.749
Epoch: 100, Train Loss: 0.008, Val Acc: 0.753
Epoch: 120, Train Loss: 0.007, Val Acc: 0.760
Epoch: 140, Train Loss: 0.007, Val Acc: 0.738
Epoch: 160, Train Loss: 0.007, Val Acc: 0.745
Epoch: 180, Train Loss: 0.006, Val Acc: 0.753
Epoch: 200, Train Loss: 0.006, Val Acc: 0.753
Test Acc: 0.707
```

GCN on Node Classification

```
from torch_geometric.nn import GCNConv
import torch.nn.functional as F

class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = GCNConv(dataset.num_node_features, 16)
        self.conv2 = GCNConv(16, dataset.num_classes)

    def forward(self, data):
        x, edge_index = data.x, data.edge_index

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        output = self.conv2(x, edge_index)

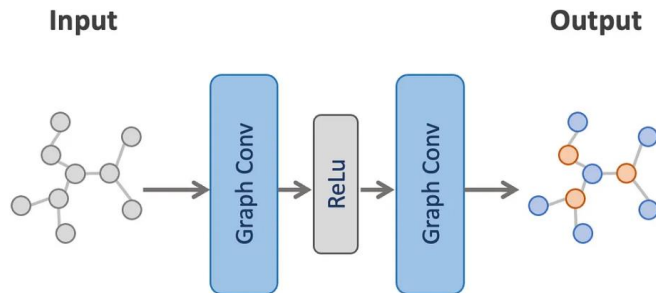
        return output

gcn = GCN().to(device)
optimizer_gcn = torch.optim.Adam(gcn.parameters(), lr=0.01, weight_decay=5e-4)
criterion = nn.CrossEntropyLoss()
gcn = train_node_classifier(gcn, graph, optimizer_gcn, criterion)

test_acc = eval_node_classifier(gcn, graph, graph.test_mask)
print(f'Test Acc: {test_acc:.3f}')
```

✓ 1.1s

```
Epoch: 020, Train Loss: 0.422, Val Acc: 0.897
Epoch: 040, Train Loss: 0.180, Val Acc: 0.926
Epoch: 060, Train Loss: 0.136, Val Acc: 0.919
Epoch: 080, Train Loss: 0.119, Val Acc: 0.919
Epoch: 100, Train Loss: 0.104, Val Acc: 0.911
Epoch: 120, Train Loss: 0.094, Val Acc: 0.915
Epoch: 140, Train Loss: 0.086, Val Acc: 0.915
Epoch: 160, Train Loss: 0.080, Val Acc: 0.915
Epoch: 180, Train Loss: 0.075, Val Acc: 0.919
Epoch: 200, Train Loss: 0.071, Val Acc: 0.919
Test Acc: 0.869
```



Books

